

Abstraction

Mladen Ivkovic

**COMP2221 Programming Paradigms –
Object-oriented Programming**



Durham
University

Join by
Web

PollEv.com
/mladenivkovic356



In This Lecture...

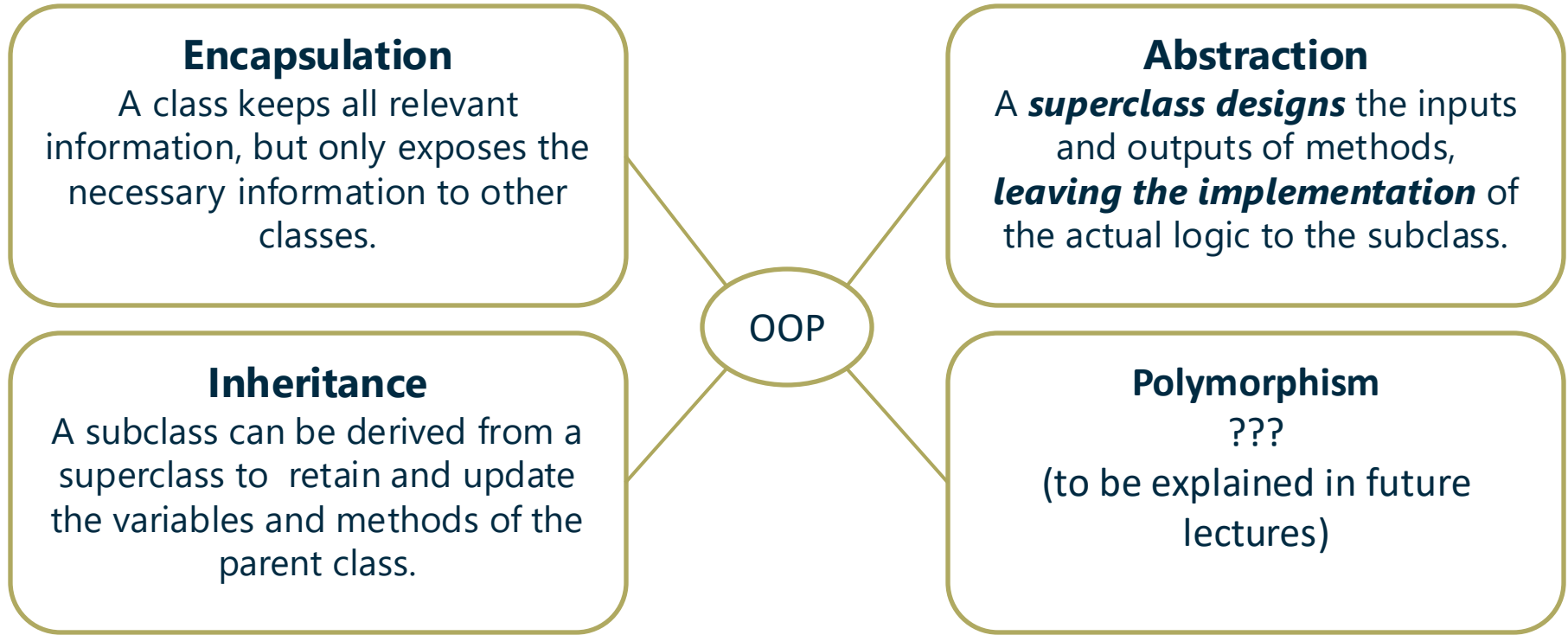
What Will You Learn in this Lecture?

- What is abstraction?
- How to code with abstraction?
 - Abstract classes, abstract methods
 - Interfaces
- A little more on UML diagrams



Abstraction

The Four Principles of OOP



Abstraction

- **Abstraction** in Java allows a superclass or interface to **define the structure** (= method signatures) while **leaving the actual implementation** to subclasses:
- Superclass implements only the **high-level design** (e.g. input/output of an abstract method) without knowing the specific logic that will be responsibility of the subclass
- This ensures that different objects can **share a common interface** (behaviour, standard) without revealing the implementation details.

Abstraction vs. Encapsulation

- **Abstraction:**

- Defines **what** a class should do but **not how**.
- **Hides implementation details** by providing method signatures without actual implementations in abstract classes or interfaces.
- **The child class must implement the details.**

- **Encapsulation:**

- **Hides internal data** and **restricts access** to it using access modifiers (private, protected, public).
- Ensures that **only necessary parts of an object** are accessible from outside.

Comparison: Encapsulation - Hiding Data

```
class BankAccount {  
    private double balance; // Hidden data (Encapsulation)  
  
    public BankAccount(double balance) {  
        this.balance = balance;  
    }  
  
    public double getBalance() { // Controlled access  
        return balance;  
    }  
  
    public void deposit(double amount) {  
        // Controlled modification  
        if (amount > 0) {  
            balance += amount;  
        }  
    }  
}
```

Encapsulation ensures that `balance` **cannot be accessed directly** from outside the class.

Comparison: Abstraction (Hiding Implementation)

```
abstract class Animal {  
    // Abstract method (Abstraction)  
    abstract void makeSound();  
  
    void sleep() {  
        System.out.println("Sleeping...");  
    }  
}  
  
class Dog extends Animal {  
    // Child class implements abstract method  
    void makeSound() {  
        System.out.println("Bark!");  
    }  
}
```

Abstraction ensures that **all Animals** must have `makeSound()`, but **each subclass defines how**.

Abstraction

- **Why use abstraction?**

- Reduces complexity
- Enhances code reusability
- Improves maintainability

- **Another Example of Abstraction:**

- A **Car** object: The user interacts with `drive()`, but doesn't need to know how the engine works.

Abstraction vs. Encapsulation: Key Differences

Key Differences

Feature	Abstraction	Encapsulation
Focus	Hides implementation details	Hides internal data
How?	Uses abstract classes and interfaces	Uses private/protected fields and public methods
Purpose	Ensures a class follows a structure without enforcing implementation	Controls how data is accessed or modified
Example	<code>abstract</code> methods that subclasses must define	<code>private</code> fields with <code>getters/setters</code>

How to Code with Abstraction?

- In Java, abstraction is implemented using:
 - **Abstract Classes**
 - **Interfaces**
- Before Java 8, Abstract Classes and Interfaces were very different. Interfaces did not provide implementations before Java 8.

Abstract Classes

- Declared using **abstract** keyword.
- **Cannot** be instantiated.
- Can contain both **abstract methods** (without a body) and **concrete methods**.
- Used when classes share common behaviour, but some methods should be implemented by subclasses.

```
abstract class Animal {  
    abstract void makeSound(); // Abstract method (no body)  
  
    void eat() { // Concrete method  
        System.out.println("This animal is eating.");  
    }  
}
```

Abstract Methods

- Defined in an abstract class.
- **Must be implemented** by any concrete subclass.
 - = must be implemented by any subclass we want to instantiate objects of
 - Implementation can be "postponed" if subclass is also abstract class itself.
- **No method body** in the abstract class.

```
abstract class Animal {  
    abstract void makeSound(); // Abstract method  
  
    void eat() { // Concrete method  
        System.out.println("This animal is eating.");  
    }  
}
```

```
class Dog extends Animal {  
    void makeSound();  
        System.out.println("Bark!");  
    }  
}
```

Variables in Abstract Classes

- Work the same way as for concrete classes!
 - Including modifiers such as `static`, `final`, `private`, `public`, `protected`
- Java does not have explicit abstract variables.

```
abstract class MyAbstractClass{
    final protected static int whatever = 1;
    abstract void printInfo();
}

class MyConcreteClass extends MyAbstractClass{
    void printInfo(){
        System.out.println(whatever);
    }
}
```

Interfaces

- Interfaces are an abstract type that define a **contract** (= set of methods) that classes must follow (=implement).
- ***All methods*** in Interfaces are abstract (before Java 8).
 - After Java 8: static methods allowed too.
- **Example:**

```
interface Animal {  
    void makeSound();  
}  
  
class Dog implements Animal {  
    public void makeSound() {  
        System.out.println("Dog barks");  
    }  
}
```

Methods & Variables in Interfaces

- **Methods:**

- `abstract` by default
- `public` by default
- No constructors allowed
- `static` concrete methods allowed (after Java 8)

- **Variables:**

- `public` only
- `static` allowed
- `final` allowed

Interfaces as a Type

- You can use Interfaces the same way you use types!
- (Full example on next slide)

```
interface Swimmable {
    void printWaterDepth();
}

class Vehicle {
    int nrPassengers;
}

class Boat extends Vehicle implements Swimmable {
    float depth;
    public void printWaterDepth() {
        System.out.println("Depth below:"+depth);
    }
}
```

```
class Animal {
    String name;
}

class Fish extends Animal implements Swimmable {
    float depth;
    public void printWaterDepth() {
        System.out.println("Fish depth:"+depth);
    }
}
```

- You can use Interfaces the same way you use types!
- Note the use of `swim()` here: ***We don't care what it is, only what it does***

```
interface Swimmable {
    void printWaterDepth();
}

class Vehicle {
    int nrPassengers;
}

class Boat extends Vehicle implements Swimmable {
    float depth;
    public void printWaterDepth() {
        System.out.println("Depth below:"+depth);
    }
}

class Animal {
    String name;
}

class Fish extends Animal implements Swimmable {
    float depth;
    public void printWaterDepth() {
        System.out.println("Fish depth:"+depth);
    }
}
```

```
public class MyMain {

    // Using interface as a type
    public static void swim(Swimmable s) {
        s.printWaterDepth();
    }

    public static void main(String[] args) {

        Boat b = new Boat();
        swim(b);

        Fish f = new Fish();
        swim(f);

    }
}
```

Interfaces and static methods

- Interfaces may have **static** methods
- Call using interface name, not object of subclass
- They are not inherited by implementing classes
- After Java 8

```
interface SaysHello {  
    static void helloWorld() {  
        System.out.println("Hello World!");  
    }  
}  
  
public class TestInterfaces {  
    public static void main(String[] args) {  
        SaysHello.helloWorld();  
    }  
}
```

Interfaces and default methods

- Interfaces may have **default** methods
 - Use keyword `default`
 - *(If no keyword is specified, abstract is assumed)*
- They are inherited by implementing classes
- They can be overridden by implementing classes
- After Java 8

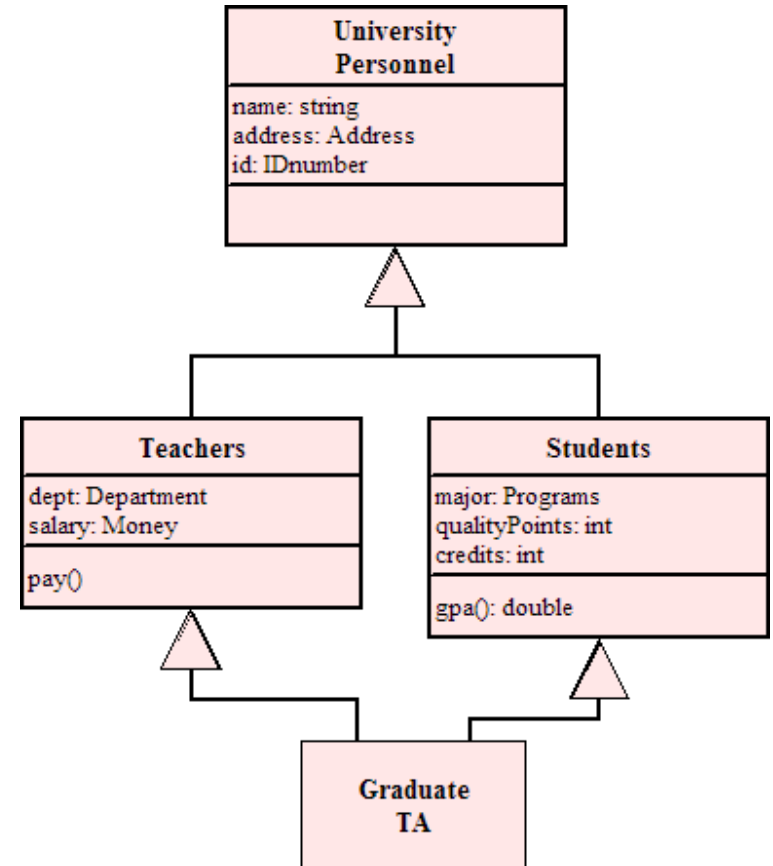
```
interface SaysHello {  
    default void helloWorld() {  
        System.out.println("Hello World!");  
    }  
}  
  
class Person implements SaysHello {  
    // lots of other code...  
}  
  
public class TestInterfaces {  
    public static void main(String[] args) {  
  
        Person p = new Person();  
        p.helloWorld();  
    }  
}
```

More about default methods

- default methods **can be overridden** by the implementing class, if necessary.
 - static methods are not inherited and cannot be overridden!
- If two or more interfaces provide a default implementation for the same method, the implementing class must provide its own implementation to avoid ambiguity.

Multi-Inheritance

- Multiple inheritance means that a subclass can inherit from two or more superclasses.
- Java **does not** allow multiple inheritance of classes:
 - Java allows **only single inheritance** of classes.
 - a subclass can inherit only one superclass.
 - Other languages, e.g. C++, allow multiple inheritance.



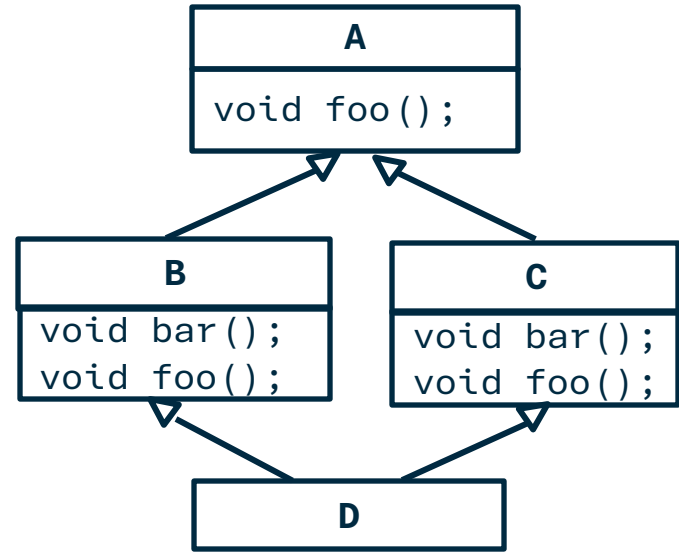
Multi-Inheritance in Java

- Instead, Java provides **Interfaces** to achieve multi-inheritance.

```
interface Flyable {  
    void fly();  
}  
  
interface Swimmable {  
    void swim();  
}  
  
class Duck implements Flyable, Swimmable {  
    public void fly() { System.out.println("Duck swims."); }  
    public void swim() { System.out.println("Duck swims."); }  
}
```

Multi-Inheritance in Java

- Why is multi-inheritance allowed for Interfaces, but not classes?
- **Design decision:** Other languages (e.g. C++) allow it.
 - Reduces code complexity, complications, ambiguities...
- **Diamond problem:** Ambiguous inheritance
- Interface methods are abstract. Since inheriting/implementing class will need to implement them, there is no ambiguity



Diamond problem – which foo(), which bar () should class D inherit?

Abstract Classes vs. Interfaces in Java

Feature	Abstract Class	Interface
Method Implementation	Can have both abstract and fully implemented methods	Can have abstract methods, default methods (with implementation), and static methods (with implementation)
Constructors	Can have constructors	Cannot have constructors
Instance Variables	Can have instance variables (fields)	Only static and final constants (by default)
Multiple Inheritance Support	No multiple inheritance (can only extend one class)	Yes , a class can implement multiple interfaces
Access Modifiers	Can have <code>public</code> , <code>protected</code> , <code>private</code> , or <code>default</code> methods	Methods are public by default (but Java 9 added <code>private</code> methods for internal reuse)
Use Case	Used when different classes share common behavior and fields	Used for defining contracts that multiple classes can implement

Which One Should You Use?

Use an abstract class when:

- You need to **share code** (variables+ methods) among related classes.
 - In particular: To share **state** (variables), as interfaces don't allow variables
- You have **partially implemented behaviour** that subclasses will *extend*.

Use an interface when:

- You need to **enforce a contract** across **unrelated** classes.
- You want to achieve **multiple inheritance**
 - Java allows only single inheritance through classes
- You need **default behaviour** but don't require shared instance variables.

Final Thoughts

- **Pre-Java 8: Abstract classes** were preferred for shared logic, and **interfaces** were strictly used for defining contracts.
- **Post-Java 8: Interfaces can have default and static methods**, making them **more powerful** and reducing the need for abstract classes in some cases.
- **Java 9+:** Interfaces can also have **private methods**, further narrowing the gap.

UML Class Diagrams

We Have Already Learned

- Association 
 - A class (container) creates an object of another class (content)
- Composition 
 - A more specific expression of association
 - A class (container) creates an object of another class (content), and the content is destroyed when the container is destroyed
- Aggregation 
 - A more specific expression of association
 - A class (container) creates an object of another class (content), and the content is NOT destroyed when the container is destroyed
- Inheritance 
 - A child class inherits a parent class

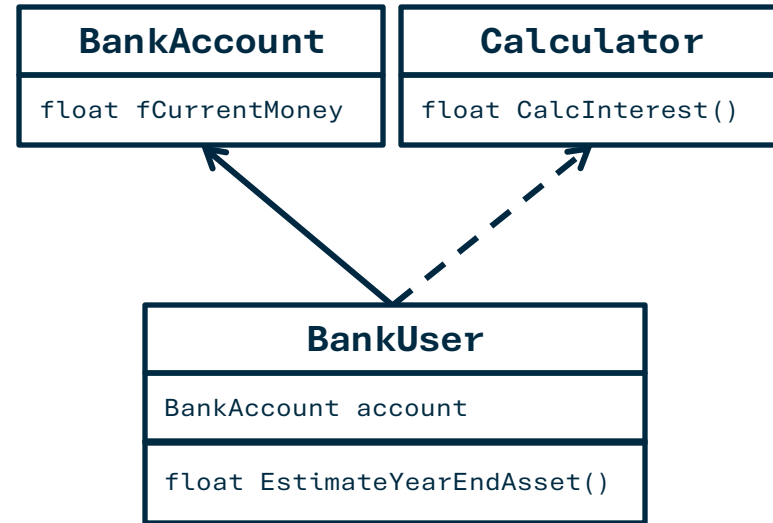
UML Arrows - Dependency

- Dependency 
 - A class uses another class but without creating an object of it

- Example:

```
public class BankUser {  
  
    private BankAccount account;  
  
    public float EstimateYearEndAsset(Calculator calc) {  
        return account.fCurrentMoney +  
            calc.CalcInterest(account);  
    }  
}
```

- BankUser has BankAccount (i.e. association, composition)
- BankUser references Calculator (i.e. dependency)



Summary

- Abstraction
- Abstract classes
- Interfaces
- More on UML class diagram



The End

Any Questions?



